

CLO	Tiêu chí đánh giá (Criteria)	Mức 1 & 2 (Không đạt/Yếu)(0 - 5.4 đ)	Mức 3 (Đạt/Trung bình)(5.5 - 6.9 đ)	Mức 4 (Khá)(7.0 - 8.4 đ)	Mức 5 (Xuất sắc)(8.5 - 10 đ)
CLO 1: Mô hình hóa cơ sở dữ liệu và luồng xử lý nghiệp vụ phía server dựa trên các yêu cầu bài toán thực tế (C4, P3)	1.1. Cấu trúc Entity & Migration (Đánh giá code first, PK/FK)	Không tạo được DB hoặc tạo sai kiểu dữ liệu , thiếu khóa chính/khóa ngoại	Sinh ra được DB từ EF Core nhưng còn lỗi nhỏ về kiểu dữ liệu. Có khóa chính tự tăng.	Các bảng map chuẩn xác. Tạo thành công quan hệ khóa ngoại qua Migration.	Code Entity gọn gàng, naming convention chuẩn. Migration sinh ra không có cảnh báo dư thừa.
	1.2. Bóc tách nghiệp vụ quan hệ n-n (Đánh giá tư duy phân tích)	Bố trí sai hoàn toàn quan hệ giữa Doanh nghiệp và Sản phẩm.	Tạo được bảng trung gian cho quan hệ n-n nhưng thiếu trường "số lượng".	Thiết lập đúng bảng n-n và có trường quản lý số lượng.	Cấu hình đúng Fluent API hoặc Data Annotations cho bảng trung gian, ràng buộc chặt chẽ.
CLO 2: Xây dựng các dịch vụ web (RESTful API) đáp ứng đúng yêu cầu nghiệp vụ và bảo đảm tính toàn vẹn dữ liệu. (C5, P3)	2.1. Cấu trúc mã nguồn & Dependency Injection	Viết toàn bộ code trong Controller, không dùng DI. Vi phạm naming convention.	Chia được thư mục, khai báo được interface/service nhưng inject sai cách hoặc còn rác code.	Áp dụng đúng DI , tổ chức thư mục chuẩn. Đặt tên class đúng format yêu cầu.	Tổ chức code cực kỳ sạch, sử dụng kế thừa hợp lý , tuân thủ tuyệt đối PascalCase/camelCase.
	2.2. Chức năng CRUD & Toàn vẹn dữ liệu	Các API bị lỗi 500. Không truyền nhận được DTO.	API Thêm/Sửa/Xóa chạy được nhưng không kiểm tra trùng lặp mã số thuế/tên.	Hoàn thiện CRUD. Có check trùng và bắt validate model bằng annotation (độ dài, bắt buộc).	Xử lý Trim() chuỗi tự động. Xử lý xóa an toàn đối với bảng có khóa ngoại.
	2.3. Xử lý ngoại lệ (Exception Handling)	API crash vắng lỗi hệ thống ra response. Trả về sai chuẩn IActionResult.	Bắt được try-catch nhưng chưa ném ra được UserFriendlyException như yêu cầu.	Áp dụng UserFriendlyException cho các lỗi nghiệp vụ (vd: trùng mã).	Bọc Exception khéo léo (có thể qua Middleware), response trả về cấu trúc JSON thống nhất, thân thiện.
CLO 3: Tối ưu hóa các luồng truy vấn dữ liệu thông qua kỹ thuật phân trang và xử lý bài toán N+1 bằng Entity Framework Core. (C5, P3)	3.1. Truy vấn phân trang & Tìm kiếm	Không viết được hàm phân trang hoặc code bằng vòng lặp thủ công.	Phân trang được nhưng tải toàn bộ DB lên RAM rồi mới .Skip().Take(). Lọc Keyword sai.	Dùng hàm LINQ đúng chuẩn để phân trang và lọc ở mức CSDL. Nhận đúng đầu vào PageSize, pageIndex.	Viết query phân trang tối ưu, tái sử dụng tốt. Code tìm kiếm xử lý hoa/thường, có loại bỏ khoảng trắng.
	3.2. Truy vấn thống kê & Tối ưu N+1	Không làm được API lấy danh sách sản phẩm nhập nhiều nhất hoặc dính lỗi N+1 Query.	API chạy ra kết quả nhưng query lồng nhau, hiệu năng kém khi test dữ liệu mẫu.	Trả ra đúng danh sách sản phẩm (mã, tên) dựa trên id doanh nghiệp, không dính lỗi truy vấn N+1.	Sử dụng linh hoạt .Include(), .Select(), GroupBy. Query sinh ra từ EF Core (xem qua log) là truy vấn SQL tối ưu nhất.